

Hybrid Cloud Setup Using VPN or Direct Connect

Project: Hybrid Cloud Setup Using VPN or Direct Connect

Services: AWS Site-to-Site VPN, Amazon VPC, Amazon EC2, Route Tables, VirtualBox

Description: Implement a hybrid cloud environment by connecting an on-premises simulated infrastructure using VirtualBox with AWS Cloud through AWS Site-to-Site VPN. Configure secure communication between local infrastructure and AWS resources using VPN connectivity, VPC networking, and route table configuration.

Skills: Hybrid cloud networking, routing, VPN configuration, secure connectivity, AWS networking, hybrid DNS concepts.

Definition: The Hybrid Cloud Setup Using VPN project is designed to establish secure communication between an on-premises simulated environment and AWS Cloud infrastructure. It uses AWS Site-to-Site VPN to securely connect a locally hosted Ubuntu virtual machine running inside VirtualBox with AWS resources deployed inside an Amazon VPC. The project demonstrates enterprise hybrid cloud networking architecture by integrating local infrastructure with AWS Cloud using secure VPN tunnels, routing mechanisms, and cloud networking services.

Scope of the Project: The scope of this project is to design and implement a secure hybrid cloud architecture by connecting a simulated on-premises environment with AWS Cloud infrastructure using AWS Site-to-Site VPN. The project focuses on secure communication, routing configuration, and connectivity between local infrastructure and cloud resources.

This project covers:

- **Simulated On-Premises Environment:** Creates a local enterprise-like infrastructure using VirtualBox and Ubuntu Server virtual machine.
- **AWS Cloud Networking:** Configures Amazon VPC, Subnets, Internet Gateway, and Route Tables for cloud-side infrastructure.
- **EC2 Deployment:** Launches Amazon EC2 instances inside AWS VPC to act as cloud resources.
- **Secure VPN Connectivity:** Establishes encrypted Site-to-Site VPN tunnels between local infrastructure and AWS Cloud.
- **Route Table Configuration:** Configures routing rules to enable communication between local and cloud networks.
- **Hybrid Cloud Integration:** Demonstrates secure hybrid networking architecture between on-premises and AWS environments.

Methodology:

1.Site-to-Site VPN + Amazon VPC – Best Method

- Uses all required AWS services including Site-to-Site VPN, VPC, EC2, and Route Tables.
- Provides secure encrypted communication between on-premises and AWS cloud environments.
- Demonstrates enterprise-level hybrid cloud architecture and networking concepts.
- Enables secure routing and communication between local and cloud infrastructure.
- Production-ready architecture suitable for organizations and enterprise networking environments.
- Cost-effective and suitable for educational and testing environments.
- Provides practical understanding of VPN tunneling, routing, and hybrid cloud integration.

2.AWS Direct Connect – Alternative Method

- Uses dedicated private connectivity between on-premises infrastructure and AWS Cloud.
- Provides lower latency and high-speed connectivity compared to internet-based VPN.
- Suitable for enterprise environments requiring dedicated and stable network communication.
- Requires physical networking infrastructure and dedicated connectivity setup.
- Involves higher operational and implementation cost compared to Site-to-Site VPN.
- Best suited for large-scale enterprise production deployments.

AWS Services used in the Project:

1.Amazon VPC (Virtual Private Cloud)

Definition: Amazon VPC is a cloud networking service that allows users to create isolated virtual networks within AWS Cloud.

Purpose: Provides secure cloud networking infrastructure for EC2 instances and VPN connectivity.

Role: Hosts AWS cloud resources, manages networking, enables VPN integration, and controls traffic routing using route tables.

2.Amazon EC2 (Elastic Compute Cloud)

Definition: Amazon EC2 is a cloud computing service that provides scalable virtual servers within AWS Cloud.

Purpose: Hosts cloud-side workloads and testing resources.

Role: Acts as the AWS cloud server that communicates with the on-premises virtual machine through VPN connectivity.

3.AWS Site-to-Site VPN

Definition: AWS Site-to-Site VPN is a managed AWS service that securely connects on-premises networks with AWS Cloud using encrypted VPN tunnels.

Purpose: Provides secure communication between local infrastructure and AWS Cloud resources.

Role: Establishes encrypted VPN tunnels between the local VirtualBox Ubuntu virtual machine and AWS VPC environment.

4.Route Tables

Definition: Route Tables contain routing rules that determine how network traffic is directed inside AWS VPC.

Purpose: Controls communication between local and cloud networks.

Role: Routes traffic between AWS VPC and on-premises network through the Virtual Private Gateway.

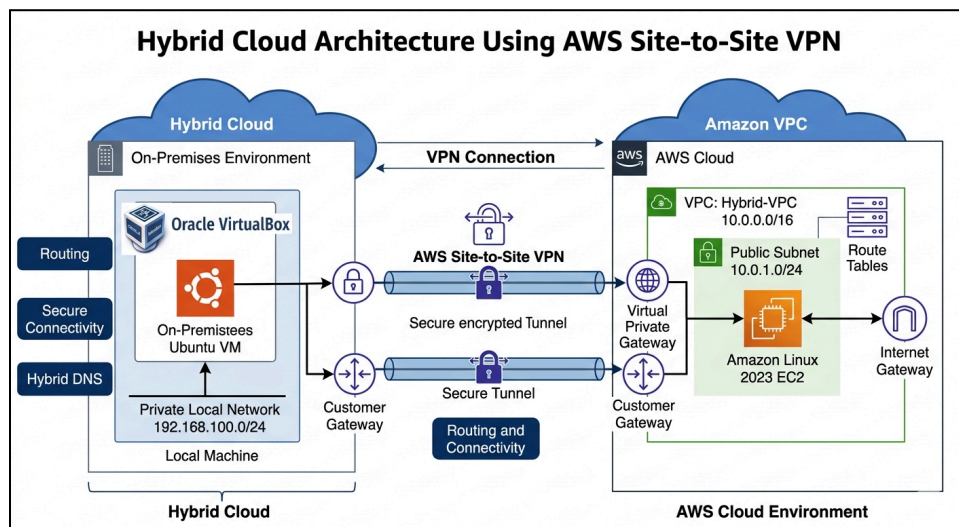
5.VirtualBox

Definition: VirtualBox is a desktop virtualization software used to create and run virtual machines on local systems.

Purpose: Simulates the on-premises enterprise infrastructure environment.

Role: Hosts Ubuntu virtual machine used for local network simulation and hybrid cloud connectivity testing.

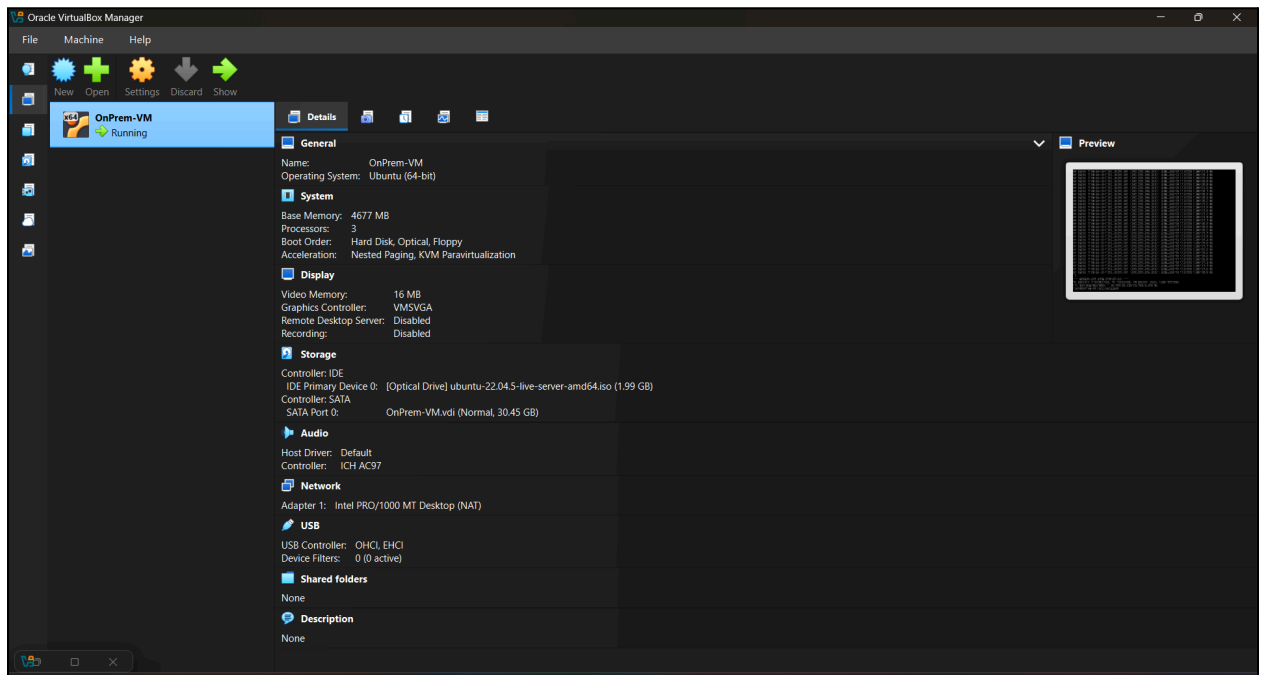
Architecture Diagram:



Implementation Steps to Do the Project:

Step1:. Create On-Premises Environment Using VirtualBox

- Install Oracle VirtualBox on the local machine.
- Download Ubuntu Server ISO image.
- Create Ubuntu virtual machine inside VirtualBox.
- Configure VM networking settings.
- Verify internet connectivity inside Ubuntu virtual machine.



```
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=10 ttl=255 time=17.3 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=11 ttl=255 time=19.1 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=12 ttl=255 time=20.3 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=13 ttl=255 time=18.8 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=14 ttl=255 time=18.8 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=15 ttl=255 time=21.5 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=16 ttl=255 time=18.7 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=17 ttl=255 time=18.0 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=18 ttl=255 time=18.0 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=19 ttl=255 time=18.2 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=20 ttl=255 time=17.2 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=21 ttl=255 time=17.0 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=22 ttl=255 time=17.0 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=23 ttl=255 time=17.2 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=24 ttl=255 time=17.4 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=25 ttl=255 time=17.7 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=26 ttl=255 time=18.0 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=27 ttl=255 time=18.0 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=28 ttl=255 time=18.2 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=29 ttl=255 time=17.7 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=30 ttl=255 time=17.9 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=31 ttl=255 time=19.1 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=32 ttl=255 time=19.6 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=33 ttl=255 time=17.7 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=34 ttl=255 time=20.0 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=35 ttl=255 time=19.2 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=36 ttl=255 time=18.6 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=37 ttl=255 time=16.8 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=38 ttl=255 time=17.1 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=39 ttl=255 time=17.7 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=40 ttl=255 time=17.6 ms
64 bytes from pu-in-f101.1e100.net (142.250.146.101): icmp_seq=41 ttl=255 time=18.5 ms
^C
--- google.com ping statistics ---
41 packets transmitted, 41 received, 0% packet loss, time 40075ms
rtt min/avg/max/mdev = 16.769/18.318/21.765/1.142 ms
root@OnPrem-VM:/etc/netplan# _
```

Step 2: Create Amazon VPC

- Go to AWS Console → VPC Dashboard → Create VPC.
- Configure VPC Name as Hybrid-VPC.
- Configure CIDR block as 10.0.0.0/16.
- Create Public Subnet with CIDR 10.0.1.0/24.
- Create Internet Gateway and attach it to the VPC.
- Configure Route Tables for internet connectivity.

Your VPCs (1/2)

Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv4 CIDR
Hybrid-VPC	vpc-06398610b5e87db8c	Available	-	-	Off	10.0.0.0/16
-	vpc-05dc21aee2e5398c8	Available	-	-	Off	172.31.0.0/16

vpc-06398610b5e87db8c / Hybrid-VPC

Details

VPC ID vpc-06398610b5e87db8c	State Available	Block Public Access Off	DNS hostnames Disabled
DNS resolution Enabled	Tenancy default	DHCP option set dopt-0cb000e37c39e15cd	Main route table rtb-0d7e0ef5894e898da
Main network ACL acl-04a0817840eedb599	Default VPC No	IPv4 CIDR 10.0.0.0/16	IPv6 pool -
IPv6 CIDR (Network border group)	Network Address Usage metrics	Route 53 Resolver DNS Firewall rule groups	Owner ID

Subnets (1/4)

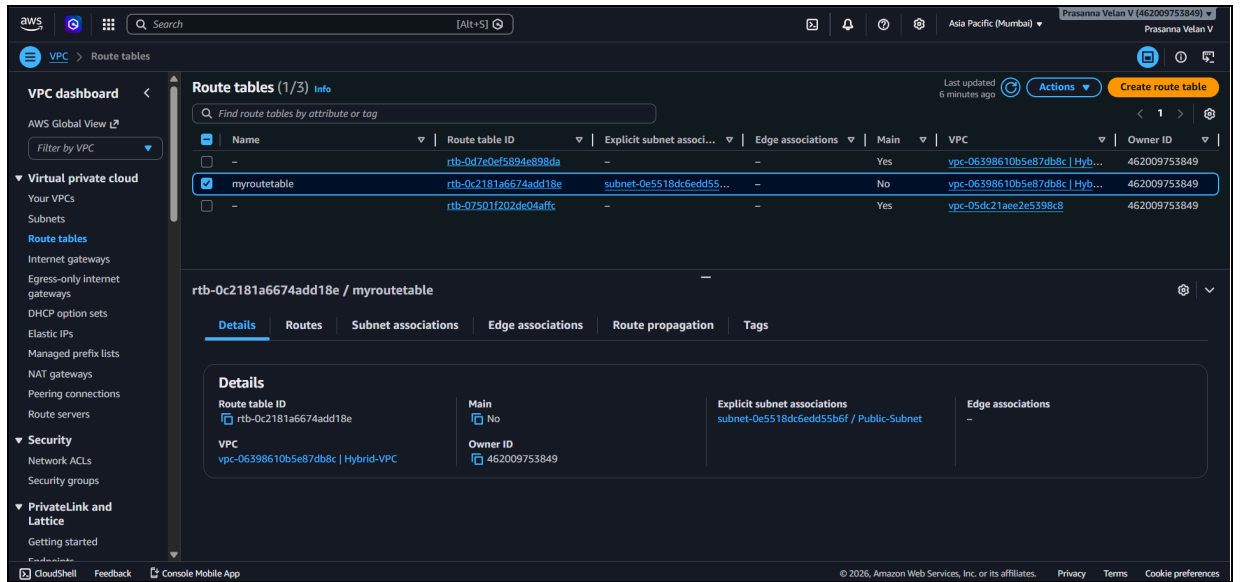
Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 C
-	subnet-04d4de6f2c6affedf4	Available	vpc-05dc21aee2e5398c8	Off	172.31.0.0/20	-
-	subnet-019af1a402542dcaa	Available	vpc-05dc21aee2e5398c8	Off	172.31.32.0/20	-
Public-Subnet	subnet-0e5518dc6edd55b6f	Available	vpc-06398610b5e87db8c Hyb...	Off	10.0.1.0/24	-
-	subnet-000ba5d5b2c8cd3e7	Available	vpc-05dc21aee2e5398c8	Off	172.31.16.0/20	-

subnet-0e5518dc6edd55b6f / Public-Subnet

Details

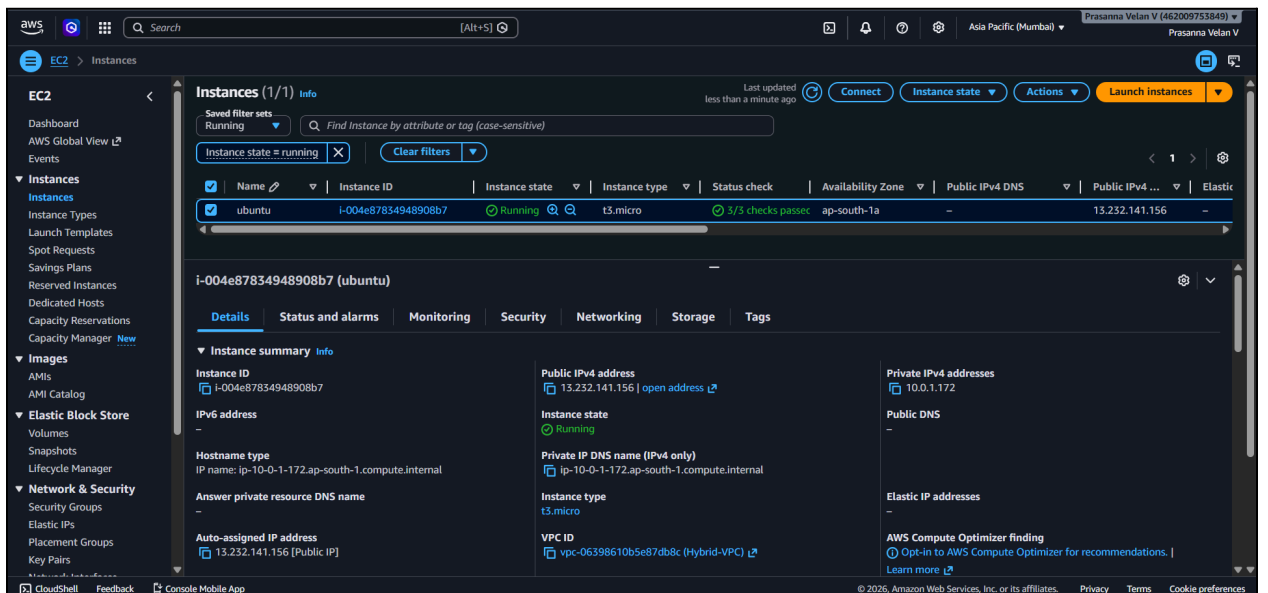
Subnet ID subnet-0e5518dc6edd55b6f	Subnet ARN arn:aws:ec2:ap-south-1:462009753849:subnet/subnet-0e5518dc6edd55b6f	State Available	Block Public Access Off
IPv4 CIDR 10.0.1.0/24	Available IPv4 addresses 250	IPv6 CIDR -	IPv6 CIDR association ID -
Availability Zone aps1-sz1 (ap-south-1a)	Network border group ap-south-1	VPC vpc-06398610b5e87db8c Hybrid-VPC	Route table rtb-0c2181a6574add18e myroutetable
Network ACL acl-04a0817840eedb599	Default subnet No	Auto-assign public IPv4 address No	Auto-assign IPv6 address No
Auto-assign customer-owned IPv4 address	Outpost ID	IPv4 CIDR reservations	

Hybrid Cloud Setup Using VPN or Direct Connect



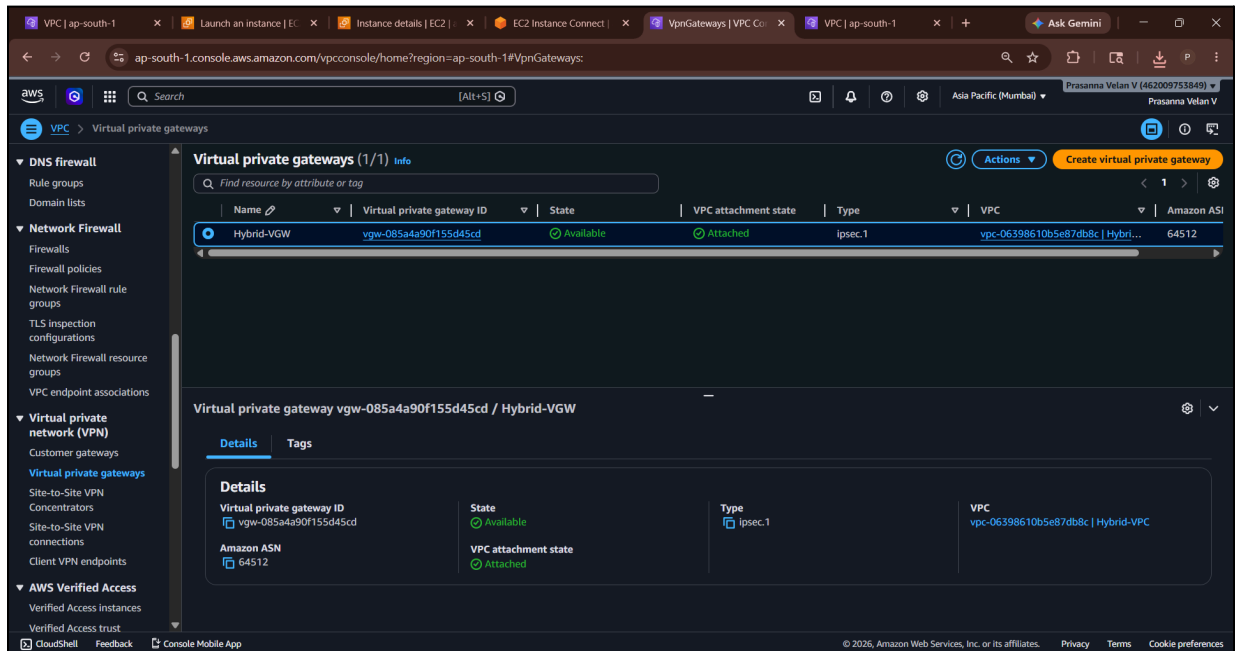
Step 3: Launch EC2 Instance

- Go to AWS Console → EC2 → Launch Instance.
- Choose Amazon Linux 2023 AMI.
- Select instance type t3.micro.
- Configure instance inside Hybrid-VPC and Public Subnet.
- Enable Auto-assign Public IP.
- Configure Security Group to allow SSH and ICMP traffic.
- Review and launch the EC2 instance.



Step 4: Create Virtual Private Gateway

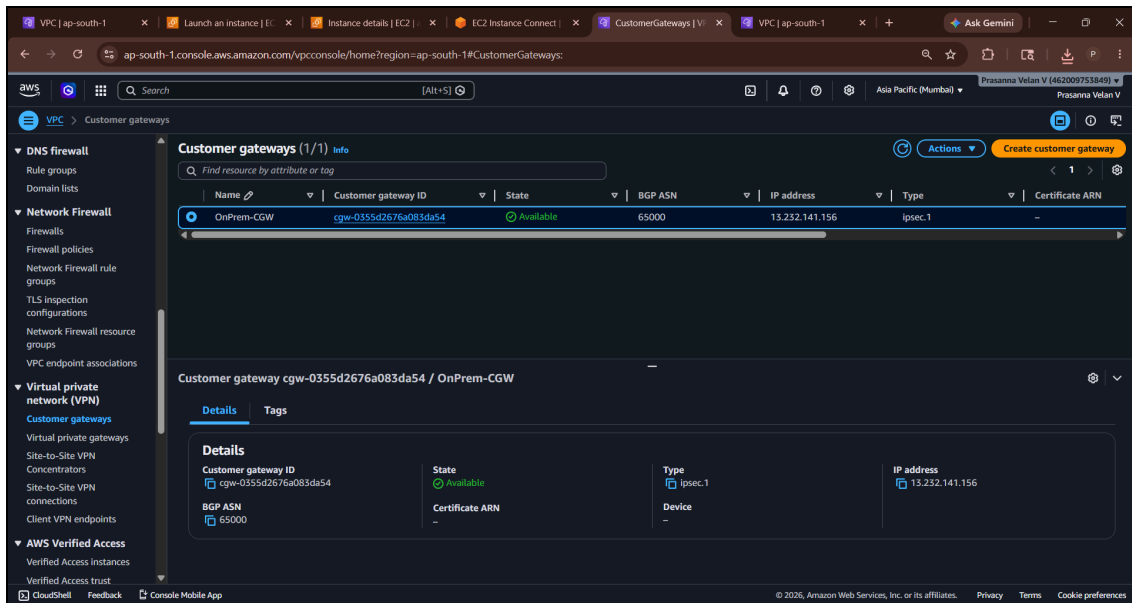
- Go to VPC Dashboard → Virtual Private Gateways.
- Create Virtual Private Gateway named Hybrid-VGW.
- Attach the gateway to Hybrid-VPC.



Step 5: Create Customer Gateway

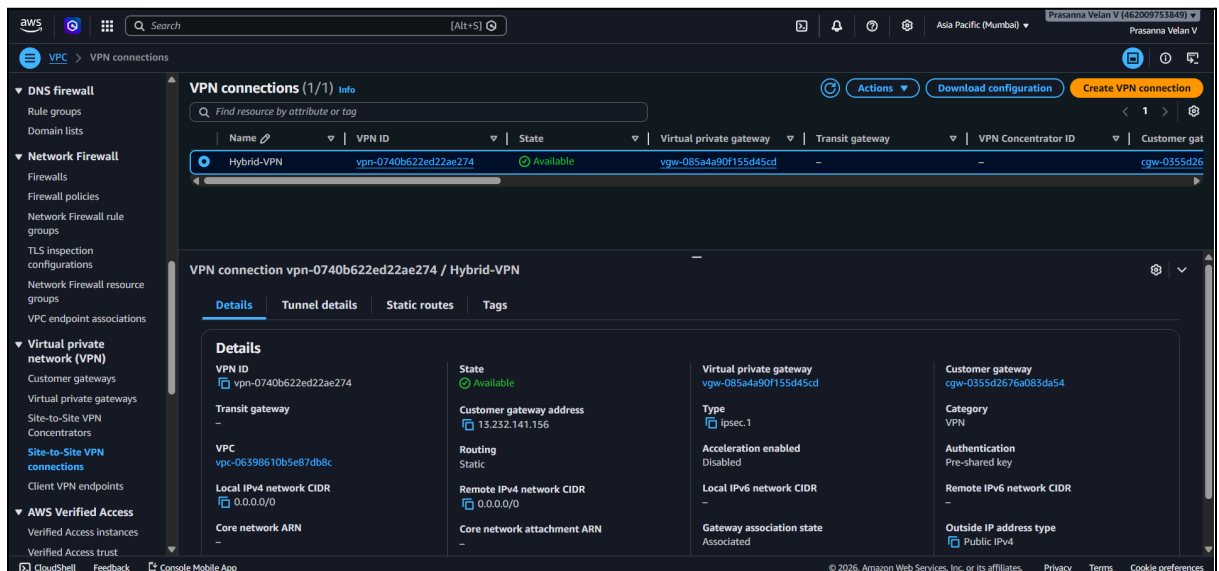
- Go to VPC Dashboard → Customer Gateways.
- Create Customer Gateway named OnPrem-CGW.
- Select Static Routing.
- Enter Public IP address of local network.

Hybrid Cloud Setup Using VPN or Direct Connect



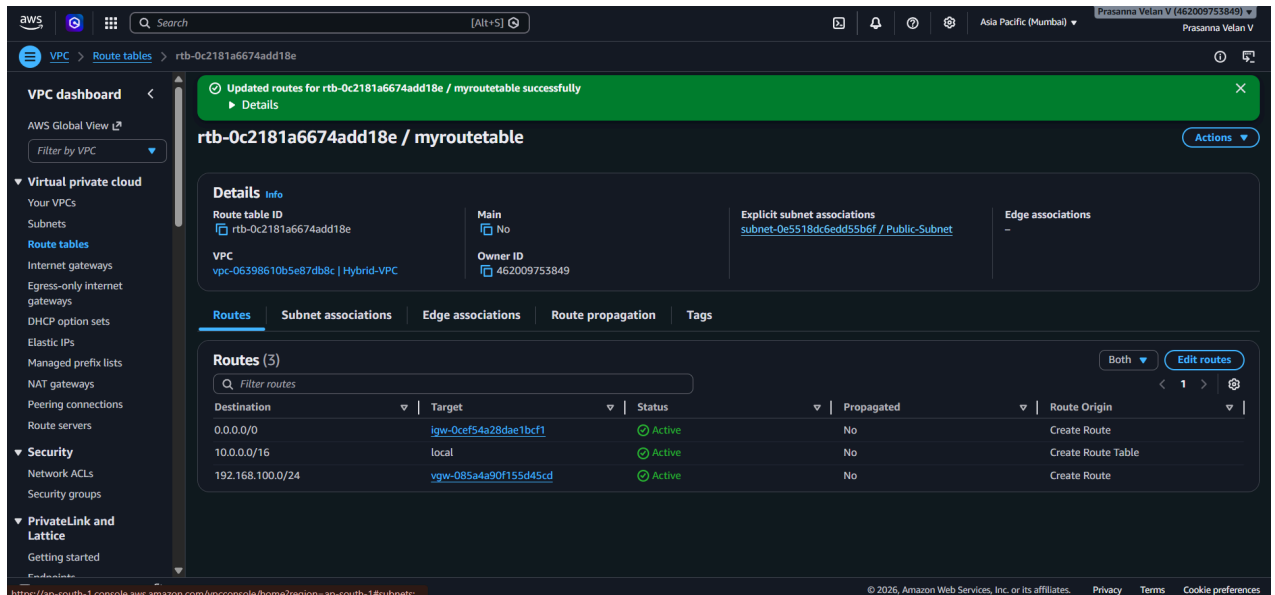
Step 6: Create Site-to-Site VPN Connection

- Go to Site-to-Site VPN Connections.
- Create VPN Connection named Hybrid-VPN.
- Select Virtual Private Gateway as target gateway.
- Select existing Customer Gateway.
- Configure Static Routing.
- Add static route 192.168.100.0/24.
- Create VPN connection.



Step 7: Configure Route Tables

- Open Route Tables inside VPC Dashboard.
- Edit routes for the VPC route table.
- Add route for 192.168.100.0/24.
- Select Virtual Private Gateway as target.
- Save route configuration.



Step 8: Download VPN Configuration

- Open Site-to-Site VPN Connections.
- Select Hybrid-VPN connection.
- Click Download Configuration.
- Select Generic vendor, platform, and software options.
- Download VPN configuration file.

Hybrid Cloud Setup Using VPN or Direct Connect

The screenshot displays the AWS Management Console for a Hybrid-VPN connection with ID `vpn-0740b622ed22ae274`. The left sidebar shows the navigation menu with categories like DNS firewall, Network Firewall, Virtual private network (VPN), and AWS Verified Access. The main content area is titled `vpn-0740b622ed22ae274 / Hybrid-VPN` and includes a 'Download configuration' button and an 'Actions' dropdown.

Details

Field	Value
VPN ID	vpn-0740b622ed22ae274
State	Available
Transit gateway	-
Customer gateway address	13.232.141.156
Routing	Static
Local IPv4 network CIDR	0.0.0.0/0
Remote IPv4 network CIDR	0.0.0.0/0
Core network ARN	-
Core network attachment ARN	-
Secrets management ARN	-
Virtual private gateway	vgw-085a4a90f155d45cd
Type	ipsec.1
Acceleration enabled	Disabled
Local IPv6 network CIDR	-
Gateway association state	Associated
VPN Concentrator ID	-
Customer gateway	cgw-0355d2676a083da54
Category	VPN
Authentication	Pre-shared key
Remote IPv6 network CIDR	-
Outside IP address type	Public IPv4

Tunnel details | Static routes | Tags

Tunnel state

Tunnel number	Outside IP address	Inside IPv4 CIDR	Inside IPv6 CIDR	Status	Provisioning status	Last status change	Details	Certificate ARN
Tunnel 1	13.204.22.101	169.254.214.52/30	-	Down	Available	May 23, 2026, 21:21:36 (UTC+05:30)	-	-
Tunnel 2	43.205.31.15	169.254.67.196/30	-	Down	Available	May 23, 2026, 21:21:36 (UTC+05:30)	-	-

```
Amazon Web Services
Virtual Private Cloud

VPN Connection Configuration
=====
AWS utilizes unique identifiers to manipulate the configuration of
a VPN connection. Each VPN connection is assigned a VPN connection identifier
and is associated with two other identifiers, namely the
customer gateway identifier and the virtual private gateway identifier.

Your VPN connection ID      : vpn-0740b622ed22ae274
Your virtual private gateway ID : vgw-085a4a90f155d45cd
Your customer gateway ID     : cgw-0355d2676a083da54

A VPN connection consists of a pair of IPsec tunnel security associations (SAs).
It is important that both tunnel security associations be configured.

IPsec Tunnel #1
=====
#1: Internet Key Exchange Configuration

Configure the IKE SA as follows:
Please note, these sample configurations are for the minimum requirement of AES128, SHA1, and DH Group 2.
Category "VPN" connections in the GovCloud region have a minimum requirement of AES128, SHA2, and DH Group 14.
You will need to modify these sample configuration files to take advantage of AES256, SHA256, or other DH groups like 2, 14-18, 22, 23, and 24.
NOTE: If you customized tunnel options when creating or modifying your VPN connection, you may need to modify these sample configurations to match the custom settings for your tunnels.

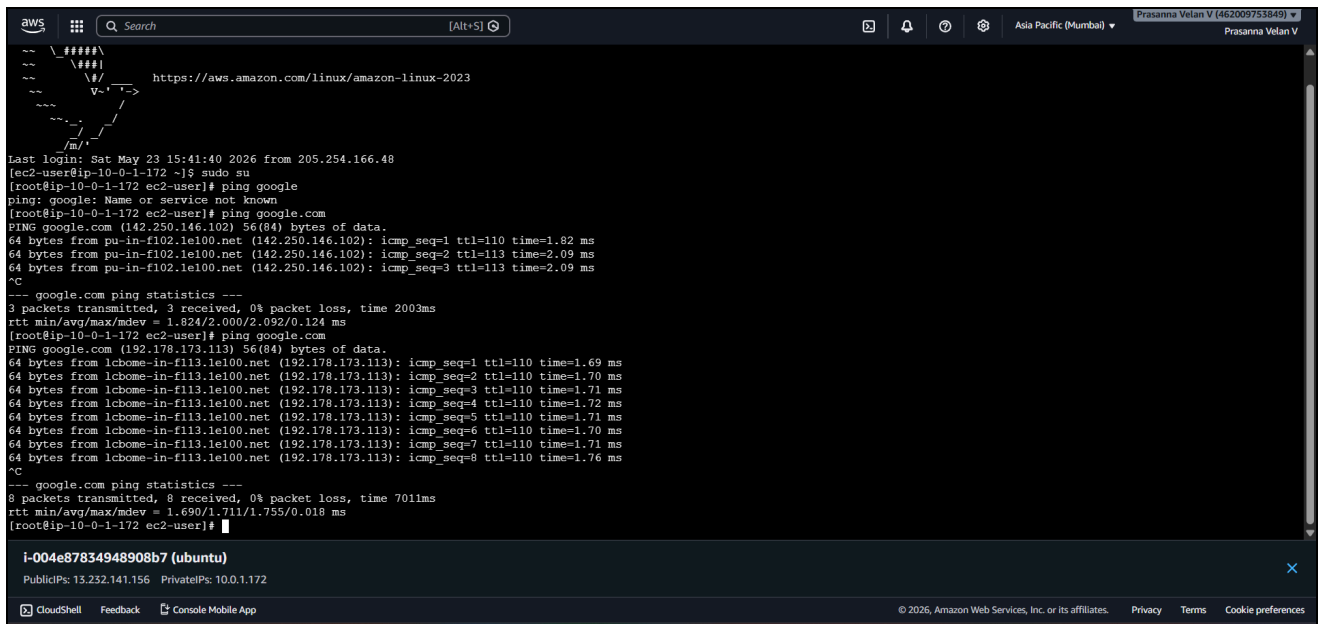
Higher parameters are only available for VPNs of category "VPN," and not for "VPN-Classic".
The address of the external interface for your customer gateway must be a static address.
Your customer gateway may reside behind a device performing network address translation (NAT).
To ensure that NAT traversal (NAT-T) can function, you must adjust your firewall rules to unblock UDP port 4500.
If not behind NAT, and you are not using an Accelerated VPN, we recommend disabling NAT-T. If you are using an Accelerated VPN, make sure that NAT-T is enabled.
- IKE version      : IKEv1
- Authentication Method : Pre-Shared Key
- Pre-Shared Key    : B4P7CejrxikMXu.n7.CofYt4KHuZwL6F
- Authentication Algorithm : sha1
- Encryption Algorithm : aes-128-cbc
- Lifetime          : 28800 seconds
- Phase 1 Negotiation Mode : main

Ln 1, Col 1    10,360 characters    Plain text    100%    Unix (LF)    UTF-8
```

Step 9: Verify Connectivity

- Verify EC2 instance status.
- Verify VPN tunnel status.
- Test internet connectivity inside Ubuntu VM.
- Validate communication between local and cloud environments.

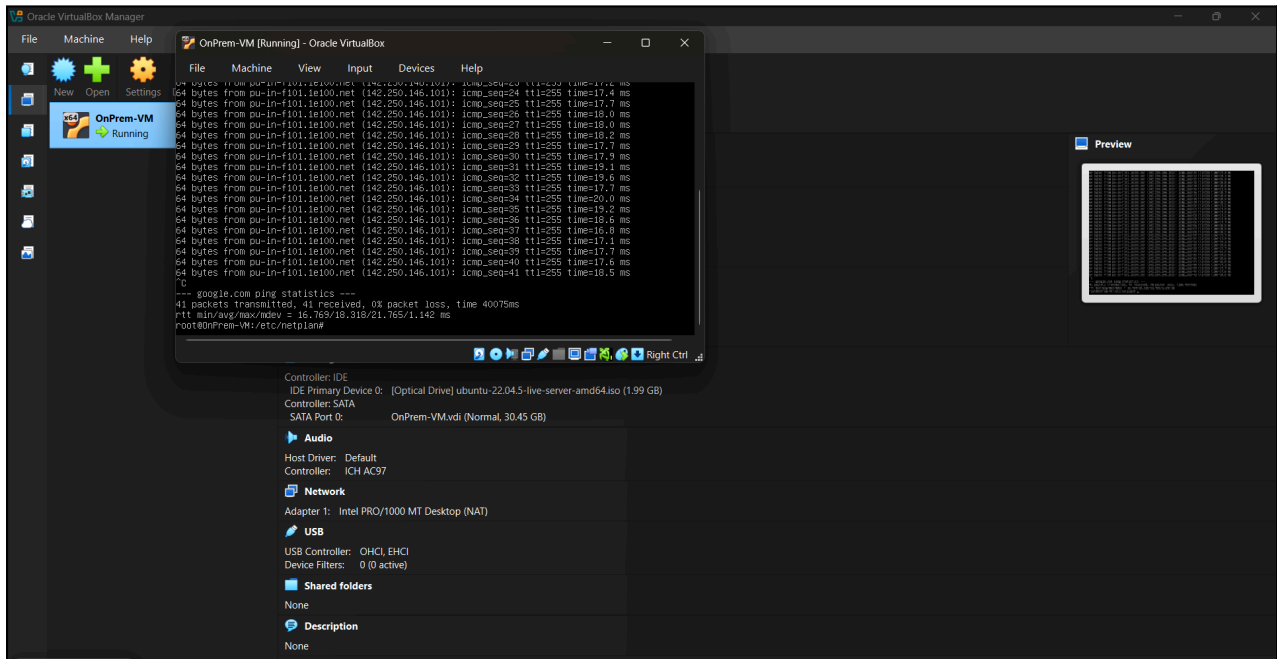
Hybrid Cloud Setup Using VPN or Direct Connect



The screenshot shows an AWS CloudShell terminal window. At the top, there's a search bar and navigation icons. The terminal output shows a user logging in and running several commands to test network connectivity. First, they run 'ping google', which fails with 'Name or service not known'. Then, they run 'ping google.com', which succeeds, showing 0% packet loss and a round-trip time of approximately 2.03ms. Next, they run 'ping google.com' again, but this time it fails with 'Name or service not known'. Finally, they run 'ping google.com' a third time, which also fails with 'Name or service not known'. The terminal output is as follows:

```
~$ https://aws.amazon.com/linux/amazon-linux-2023
Last login: Sat May 23 15:41:40 2026 from 205.254.166.48
[ec2-user@ip-10-0-1-172 ~]$ sudo su
[root@ip-10-0-1-172 ec2-user]# ping google
ping: google: Name or service not known
[root@ip-10-0-1-172 ec2-user]# ping google.com
PING google.com (142.250.146.102) 56(84) bytes of data.
64 bytes from pu-in-f102.1e100.net (142.250.146.102): icmp_seq=1 ttl=110 time=1.82 ms
64 bytes from pu-in-f102.1e100.net (142.250.146.102): icmp_seq=2 ttl=113 time=2.09 ms
64 bytes from pu-in-f102.1e100.net (142.250.146.102): icmp_seq=3 ttl=113 time=2.09 ms
^C
--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/ndev = 1.824/2.000/2.092/0.124 ms
[root@ip-10-0-1-172 ec2-user]# ping google.com
PING google.com (192.178.173.113) 56(84) bytes of data.
64 bytes from lcbome-in-f113.1e100.net (192.178.173.113): icmp_seq=1 ttl=110 time=1.69 ms
64 bytes from lcbome-in-f113.1e100.net (192.178.173.113): icmp_seq=2 ttl=110 time=1.70 ms
64 bytes from lcbome-in-f113.1e100.net (192.178.173.113): icmp_seq=3 ttl=110 time=1.71 ms
64 bytes from lcbome-in-f113.1e100.net (192.178.173.113): icmp_seq=4 ttl=110 time=1.72 ms
64 bytes from lcbome-in-f113.1e100.net (192.178.173.113): icmp_seq=5 ttl=110 time=1.71 ms
64 bytes from lcbome-in-f113.1e100.net (192.178.173.113): icmp_seq=6 ttl=110 time=1.70 ms
64 bytes from lcbome-in-f113.1e100.net (192.178.173.113): icmp_seq=7 ttl=110 time=1.71 ms
64 bytes from lcbome-in-f113.1e100.net (192.178.173.113): icmp_seq=8 ttl=110 time=1.76 ms
^C
--- google.com ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 701ms
rtt min/avg/max/ndev = 1.690/1.711/1.755/0.018 ms
[root@ip-10-0-1-172 ec2-user]#
```

At the bottom of the terminal window, there's a status bar showing the instance ID 'i-004e87834948908b7 (ubuntu)', public IP '13.252.141.156', and private IP '10.0.1.172'. There are also links for 'CloudShell', 'Feedback', and 'Console Mobile App'.



Troubleshooting:

Issue 1: VPN Tunnel Status Showing Down.

The VPN tunnel may initially display DOWN because the local VPN endpoint is not fully configured or connected.

Solution: Verify Customer Gateway public IP address and VPN configuration settings. Ensure route tables are configured correctly and the Virtual Private Gateway is properly attached to the VPC.

Issue 2: EC2 SSH Connection Failed.

SSH connection to EC2 instance may fail because of incorrect security group rules or invalid key permissions.

Solution: Allow SSH port 22 in the EC2 Security Group. Verify key pair permissions and ensure correct public IP address is used during SSH connection.

Issue 3: Route Table Configuration Error.

Incorrect route table entries may prevent communication between AWS and local networks.

Solution: Verify destination CIDR blocks and ensure the Virtual Private Gateway is selected as the correct target in route table configuration.

Conclusion:

The Hybrid Cloud Setup Using VPN project successfully demonstrates secure integration between an on-premises simulated environment and AWS Cloud infrastructure. Using AWS Site-to-Site VPN, Amazon VPC, EC2, and Route Tables, secure communication between local and cloud environments is established effectively.

The project provides practical understanding of hybrid cloud networking, VPN tunneling, routing, and secure connectivity concepts. It demonstrates enterprise-level hybrid cloud architecture and showcases how organizations can securely integrate local infrastructure with AWS Cloud services using secure networking solutions.